

■ DELIVERY DEADLINE: 15 JULY 2026

1. Project Overview

The goal is to build a white-label **AI-powered live chat widget** that any business can embed into its website. The widget answers visitor questions by querying a company-specific knowledge base. When the AI cannot confidently answer, it automatically escalates the query to the business owner via email. An intelligent onboarding flow generates tailored questions to seed the knowledge base.

Embeddable Widget

Knowledge Base RAG

Email Escalation

AI Onboarding

2. Scope & Deliverables

2.1 Admin Dashboard (SaaS Backend)

- Secure login / registration for business owners.
- Knowledge base management: upload documents (PDF, DOCX, TXT), paste website URLs for scraping, add plain-text FAQs.
- Onboarding wizard: after first upload, AI generates **10 tailored mock questions** for the owner to answer — answers are injected into the knowledge base.
- Email settings: configure escalation recipient address, custom subject prefix, and reply-to.
- Widget customiser: brand colours, avatar, welcome message, widget position.
- Conversation history & analytics dashboard (message volume, escalation rate, top topics).
- API key management for the embed snippet.

2.2 Embeddable Chat Widget

- Universal **JavaScript snippet** (vanilla JS, <2 KB gzip) that can be dropped into any HTML page.
- Official **WordPress plugin** (auto-inserts snippet via Settings page, no code required).
- Official **React component** (npm package, configurable via props).
- All three implementations must be visually and behaviourally identical.
- Responsive design — works on mobile, tablet, and desktop.
- Supports markdown in AI replies (bold, lists, links).
- Typing indicator while AI is processing.
- Optional email-capture field before first message (configurable).

2.3 AI Answer Engine (Backend)

- Retrieval-Augmented Generation (RAG) pipeline: chunk and embed uploaded documents, store vectors, retrieve top-k relevant chunks per query.
- LLM call with system prompt scoped strictly to retrieved context — the AI must not answer from general knowledge.
- Confidence threshold: if similarity score of best-matching chunk falls below a configurable threshold, trigger email escalation instead of (or alongside) a low-confidence reply.
- Streaming responses to the widget (SSE or WebSocket).

2.4 Email Escalation

- When escalation is triggered, send an email to the configured owner address containing: visitor question, conversation context (last 5 messages), timestamp, page URL.
- The owner can reply directly to the email — the reply is displayed in the visitor's chat thread (webhook-based inbound email parsing).
- Escalation events are logged in the admin dashboard.

3. AI Onboarding Flow — Detail

This is a critical differentiator. After the owner uploads their first piece of content, the system must automatically generate **10 contextually relevant mock questions** that a typical website visitor might ask. The owner answers each question in the dashboard; these Q&A pairs are stored as high-priority knowledge base entries.

Step	Action	Actor
1	Owner uploads documents / URLs	Owner
2	System processes & embeds content	Backend
3	AI generates 10 mock Q&A prompts	AI Engine
4	Owner reviews & answers each question	Owner
5	Answers are embedded & stored as KB entries	Backend
6	Widget goes live with seeded knowledge	System

4. Recommended Tech Stack

The developer is free to propose alternatives, but must justify deviations from the recommendations below.

Layer	Recommended	Notes
Frontend Widget	Vanilla JS + Web Components	Zero dependencies for universal embed
WordPress Plugin	PHP 7.4+, standard WP plugin API	Submit to WP plugin directory eventually
React Package	React 18+, published to npm	Peer-dep on React; TypeScript types required
Admin Dashboard	Next.js 14 (App Router)	Auth.js for authentication
Backend API	Python / FastAPI	Async, OpenAPI docs auto-generated

Vector Store	pgvector (PostgreSQL) or Qdrant	Self-hostable; avoid proprietary lock-in
Embeddings	OpenAI text-embedding-3-small	Or open-source via sentence-transformers
LLM	OpenAI GPT-4o (configurable)	Abstract provider — allow Claude / Gemini swap
Email	SendGrid or Resend	Inbound webhook for owner replies
Hosting	Railway / Render (backend), Vercel (frontend)	Serverless or Docker-based deploy preferred

5. Widget Integration Examples

5.1 HTML / JavaScript (Universal Snippet)

```
<script>
  window.ChatbotConfig = { apiKey: 'YOUR_API_KEY', position: 'bottom-right' };
</script>
<script src='https://cdn.yourdomain.com/widget.js' async></script>
```

5.2 WordPress

Install the plugin from the WordPress plugin directory (or upload the .zip). Navigate to **Settings → AI Chatbot**, paste the API key, and save. Done.

5.3 React / npm

```
npm install @yourcompany/ai-chatbot-widget

import { ChatWidget } from '@yourcompany/ai-chatbot-widget';

<ChatWidget
  apiKey='YOUR_API_KEY'
  position='bottom-right'
  primaryColor='#2563EB'
/>
```

6. Core API Endpoints

Method	Endpoint	Description
POST	/api/chat	Send message; returns AI reply (streaming SSE)
POST	/api/kb/upload	Upload document to knowledge base
POST	/api/kb/url	Scrape and ingest a URL
GET	/api/onboarding/questions	Fetch AI-generated onboarding questions
POST	/api/onboarding/answers	Submit owner answers to onboarding questions
GET	/api/conversations	List conversation threads (admin)
GET	/api/conversations/{id}	Get full conversation with escalation status
POST	/api/escalate/{id}	Manually trigger email escalation for a thread
POST	/api/settings	Update widget config, email, thresholds

7. Non-Functional Requirements

- **Latency:** First AI token must stream to widget within 1.5 s under normal load.
 - **Widget bundle size:** < 30 KB gzip (no heavy framework dependencies).
 - **Scalability:** Backend must handle 100 concurrent chat sessions without degradation.
 - **Security:** API keys validated on every request; rate limiting (60 req/min per key); HTTPS only.
 - **Data isolation:** Each business's knowledge base is strictly isolated — no cross-tenant leakage.
 - **GDPR:** Visitor chat logs must be deletable on request; no PII stored in vector embeddings.
 - **Browser support:** Widget must work on Chrome, Firefox, Safari, Edge (last 2 major versions).
 - **Uptime SLA:** 99.5% monthly uptime target for the chat API.
-

8. Milestones & Timeline

#	Milestone	Due Date	Deliverable
M1	Architecture & DB schema approved	30 May 2026	Architecture doc + ERD
M2	Backend API + RAG pipeline (no widget)	13 Jun 2026	Postman/OpenAPI collection
M3	Admin dashboard — MVP	20 Jun 2026	Deployed staging URL
M4	JS widget + WordPress plugin	30 Jun 2026	npm package + .zip plugin
M5	React package + email escalation	07 Jul 2026	npm publish (beta tag)
M6	Onboarding flow + QA + bugfix	12 Jul 2026	Test report
M7	Final delivery & handover	15 Jul 2026	Full source + documentation

9. Acceptance Criteria

The build is considered **complete** when ALL of the following are verified:

- AC-01** A new business can sign up, upload a document, complete the 10-question onboarding, and have a working widget in under 10 minutes.
- AC-02** The widget JS snippet, WordPress plugin, and React component all produce visually identical UIs on a reference test page.
- AC-03** AI correctly answers at least 90% of questions directly derivable from uploaded test documents (manual QA set of 20 questions).
- AC-04** Questions not answerable from context trigger an email to the configured address within 30 seconds.
- AC-05** Owner email reply appears in visitor chat thread within 60 seconds.
- AC-06** Widget loads in < 500 ms on a 4G connection (Lighthouse simulation).
- AC-07** All API endpoints are documented in OpenAPI 3.0 format.
- AC-08** Source code is delivered in a private Git repository with a README covering local setup, environment variables, and deployment.

AC-09 Unit test coverage $\geq 60\%$ on backend business logic.

10. Out of Scope (v1)

- Native mobile SDKs (iOS / Android) — planned for v2.
- Multi-language UI localisation of the admin dashboard.
- Voice / audio input in the chat widget.
- Live human agent handoff / ticketing system integration.
- Custom LLM fine-tuning.

11. Open Questions for Developer

#	Question	Status
Q1	Preferred vector store: pgvector (simpler ops) or Qdrant (better perf at scale)?	Open
Q2	Should the widget support multi-language detection automatically?	Open
Q3	Do we need a white-label/custom domain option for the admin dashboard in v1?	Open
Q4	Will LLM API keys be provided by Verdia or by each end-customer?	Open
Q5	Is there a design mockup / Figma, or does the developer have creative freedom on UI?	Open

This document is prepared by Verdia j.d.o.o. · Confidential — for assigned developer only · All questions to be addressed before kick-off meeting.